MEDICATION SAFETY SIGNAL MONITOR · HAND-IN R06

Optimization log

Version 1.2, 6 November 2026: measure, change, measure again, and say what did not help

The question this report answers

The brief asks us to pick at least one pipeline, measure it, improve it, and show before and after (build time, compute, or the cost of the LLM step), and to say what we tried that did not help. What did we measure, and what happened?

One-line answer

Two changes helped: the AI cache (model calls 9 to 0 on a rebuild) and DailyMed XML reuse (3 m 23 s to 1 m 23 s). Two did not: incremental FAERS was correct but 2.75 times slower, and column pruning made no real difference.

What this is	What this is not
• The optimization log the brief asks for (hand-in item 3), written out in full • Four entries, O1 to O3a (O1 re-checked as O1b after D041 v2), each with baseline, change, after, method and threats to validity • The deliberate choices behind them: incremental or full rebuild, partitioning, where the LLM step sits	• A claim that the pilot is fast or cheap at production scale • A cost report in dollars: we measured tokens and seconds, not invoices • A record of compute-seconds: Foundry build times are wall-clock times

How to read this. Chapter 1 is the one-page answer. Chapter 2 says how we measured and what can make a measurement wrong. Chapters 3 to 6 are one entry each. Chapter 7 is the design choices the judges look for. Chapter 8 says what we would measure next. Numbers from `numbers.json` are listed in the table at the end. Figures from other project records cite that record where they appear and are listed in *Figures quoted from project records*.

Contents	Page
01 · Summary in plain language	2
02 · How we measured	4
03 · O1 · The AI cache	5
04 · O2 · Incremental FAERS	6
05 · O3 · DailyMed XML reuse	8
06 · O3a · Column pruning	9
07 · Deliberate choices	10
08 · What we would measure next	12

01

Summary in plain language

This project runs on **Palantir Foundry**, a data platform where data pipelines, a data model and apps live together. A **dataset** is a table stored in Foundry. A **transform** is a piece of Python code that reads datasets and writes a new one; running it is a **build**. A transform can be **incremental**: it remembers what it wrote last time and only does the new work. **AIP** is Foundry's way of calling a large language model (an **LLM**, here Claude) from a pipeline.

We tried four optimizations on three pipelines. The sources are FAERS (the FDA's database of adverse event reports, published as quarterly files) and DailyMed (the official drug labels, as SPL XML, a structured label format). The pilot covers 13 drugs: 8 opioids (oxycodone, hydrocodone, morphine, fentanyl, hydromorphone, methadone, buprenorphine, tramadol) and 5 GLP-1 drugs (semaglutide, liraglutide, dulaglutide, tirzepatide, exenatide).

Each claim carries a label. Evidence labels say what kind of claim it is; status labels say how far the work has got:

Label	Meaning
OBSERVED	Read directly from a public source (FAERS, DailyMed, openFDA, RxNorm), with the query that produced it.
DERIVED	Computed by our pipeline from observed data, for example a PRR or a count per drug and reaction.
ASSUMED	Not verified: taken as true for now because the data or the brief is silent. It could be wrong.
DESIGNED	A design choice we made and wrote down (the decision log says why). It may be built; the status band says how far.
TESTED	Built, run, and checked against evidence.
PLANNED	On the plan; not built yet.
PROPOSED	Suggested; waiting for a decision.

#	Pipeline	Before	After	Helped?	Evidence
O1	AI label extraction (LLM step)	9 model calls, 70,776 tokens	0 calls, 0 tokens on rebuild	**Yes**	OBSERVED
O1b	Same cache after D041 v2 (5 labels)	Without a cache: all 21 sections re-sent each build	Widening build 12 calls (new sections only); forced rebuild 0 calls	**Yes**	OBSERVED
O2	FAERS clean: incremental by quarter	Full rebuild 32 s	Incremental 88 s (same rows)	**No**	OBSERVED
O3	DailyMed raw pull: reuse XML	3 m 23 s, 1,321 downloads	1 m 23 s, 2 downloads	**Yes**	OBSERVED
O3a	FAERS clean: parse fewer columns	Local run 12.3-13.0 s	Local run 12.2 s	**No** (reverted)	OBSERVED locally

The short version

What helped: not doing work twice. The LLM is not called again for a label section it has already read (O1), and a label XML is not downloaded again if we already hold that exact version (O3).

What did not: incremental FAERS is correct row for row but 2.75 times slower at two quarters (O2), and parsing fewer columns saved a fraction of a second (O3a). Both are kept in the log on purpose.

What this means for Dana (the safety officer)

Faster builds do not make her flags fresher. Signals change only when a new FAERS quarter arrives, and FAERS is about 130 days behind (monitoring `faers_age_days`). FAERS was replaced by AEMS (the FDA's new adverse event system) in March 2026, so how the future feed will arrive is unverified (DQ20).

A new label still reaches her the same day. XML reuse (O3) keeps pulling the DailyMed index every build, so a new label version is fetched as soon as it is listed.

The AI cache (O1, O1b) costs her nothing and hides nothing. The AI's exact quotes are on her page 1 since 6 Nov, and a label refresh with the same text costs no model call. The AI does not change her signal list (report R08).

Only the project owner can submit a triage decision today. Giving the officer that permission is a production step (report R04).

Gate: Brief requirement met TESTED

Required evidence: At least one pipeline measured, improved, with before and after; failures shown.

Result: Met twice (O1 LLM cost, re-checked as O1b at 2.3 times the size; O3 build time). Two non-gains reported with their reasons (O2, O3a).

If it fails: Re-run O3 back-to-back before quoting it as the headline number (chapter 5).

02

How we measured

Our rule, taken from the brief's judging notes: **measured, not asserted**. Every entry has a baseline, one change, and an after number taken the same way.

What	How we read it	Used for
Build time	Duration on the Foundry **Builds** page (wall-clock time of the job)	O2, O3
Model calls and tokens	Columns written by the AI transform: `from_cache`, `called_at`, `prompt_tokens`, `completion_tokens` in `ai_label_section_extractions`	O1
Same output	Foundry SQL: row counts, and `EXCEPT ALL` in **both** directions (rows in A not in B, and B not in A); both must return 0	O2
Same output (local)	Sorted row hashes from the local runner, a copy of each transform that runs on a laptop and equals Foundry (A014)	O3a
Download count	Label rows fetched in the build vs rows carried over from the previous output	O3

Threats to validity that apply to every entry

Threat	What it means here	Effect
One run each	Each before and after is a single build. We did not repeat builds to get a spread.	Small differences (a few seconds) are noise. Only large gaps are claimed.
Foundry start-up overhead	Every Foundry build spends time starting before the code runs. We did not measure it separately.	Hurts any design with more builds (O2 pays it twice).
Wall-clock, not compute	The Builds page shows elapsed time, not compute-seconds (machine time used). We did not capture compute-seconds.	We cannot state a compute or money saving.
Pilot size	Two FAERS quarters, 13 drugs, 1,321 labels, 5 labels read by the AI (21 sections).	Results may change at scale; chapter 8 says what to re-measure.

Gate: Method written down TESTED

Required evidence: Each number has a named place it was read from.

Result: Met. Compute-seconds and repeated runs are **not measured** and are said so.

If it fails: Treat the number as a guess and leave it out of the demo.

03

O1 · The AI cache

TESTED	Helped. A rebuild with unchanged label text made 0 model calls (was 9).

The pipeline. The AI transform sends each safety section of a drug label to Claude Sonnet (through AIP) and gets back every reaction it names, each with an exact quote (decision D041). O1 was measured on the first pilot: 2 oxycodone labels, 9 sections. Since D041 v2 it reads 5 labels, 21 sections (O1b below). Calling an LLM costs tokens (pieces of text the model reads and writes) and time, so doing it twice for the same text is pure waste.

	Value	Evidence
Baseline (run 1)	9 model calls; 41,704 prompt + 29,072 completion = 70,776 tokens	OBSERVED
Change	A cache. Each section gets a **cache key**: label setid + version + section + a hash of the section text + parser, prompt and model version (D005, D041). Before calling the model, the transform looks the key up in its own previous output (an incremental transform).	DESIGNED
After (rebuild after an upstream refresh, same text)	0 model calls, 0 tokens; all 9 sections served from cache; same output rows	OBSERVED
O1b (6 Nov, D041 v2: add 3 semaglutide labels)	Widening build: 12 model calls, only the new sections (44,869 prompt + 26,226 completion tokens); 9 old sections reused. Forced rebuild with unchanged text: 0 calls, 21 of 21 sections from cache	OBSERVED
Method	`from_cache` = true and `called_at` unchanged on every section row after the rebuild	OBSERVED

Threats to validity

- The job's wall time was **not** compared like for like, so we claim calls and tokens saved, not seconds saved.
- Cost is in tokens, not dollars. We did not record the AIP price per token, so a money figure is **not measured**.
- Run 1 used prompt p1. The current prompt p2 was a fresh run; it used about 55% more output tokens (A011 run 2; decision-log learning 2026-11-04). The exact p2 count is not in `numbers.json`. The cache behaviour is the same for both.

How we know the cache is safe, not just cheap

The cache key includes the prompt and model version. When we moved from prompt p1 to p2, the key changed and all sections were extracted again (A011, run 2). So the cache saves calls when nothing changed, and does **not** hide a change that should be re-scored.

O1b needed a **forced** build: the repository Build button and the dataset Build button both skipped the unchanged AI dataset as 'already up to date' (DECISIONS learning 2026-11-06). Build ▾ → Force ran it, and every section came from the cache.

Gate: O1 closed TESTED

Required evidence: Model calls and tokens before and after, same output.

Result: Met: 70,776 tokens to 0 on rebuild; O1b confirms it at 21 sections.

Measure next: Job wall time, back-to-back; dollars per label before widening the AI to other drugs.

04

O2 · Incremental FAERS

TESTED	Did not help at two quarters: 88 s incremental vs 32 s full rebuild (2.75 times slower). Outputs identical.

The pipeline. `clean_faers` turns FAERS quarter files into three clean tables: current cases, case reactions, and drug-reaction rows for our drugs. FAERS sends follow-up versions of a case in later quarters and also deletes cases, so a new quarter can change old rows (decision D022).

	Value	Evidence
Baseline	Full rebuild reads both quarters (819,683 DEMO rows, the case file, plus their drug and reaction files): **32 s**. DEMO rows count every case version; after rule D022 keeps one version per case and applies deletes, 792,346 current cases remain	OBSERVED DERIVED
Change	Incremental by quarter (A015, D044): read only the new quarter, merge it into a kept **state** with the D022 rule (highest case version wins, then the quarter's deletes), log each quarter, then rebuild the three outputs from the state	DESIGNED
After (adding Q2)	Reads one quarter (422,459 DEMO rows, 52%) plus the state: **88 s** = state step 62 s + outputs step 26 s	OBSERVED
Correct?	Yes. 792,346 cases, 2,554,017 reactions, 373,522 drug-reaction rows in both; `EXCEPT ALL` both ways = 0. Q2 replaced 26,681 Q1 cases and deleted 655	OBSERVED
Method	Foundry Builds page durations; Foundry SQL for equality	OBSERVED

Why it was slower

The saving is one quarter of file parsing. The cost is fixed: read back the saved state, write the three state datasets and the quarter log again, and run a **second** build for the outputs. Each Foundry build also pays its start-up time, and the incremental path pays it twice. At two quarters the fixed cost is bigger than the saving.

When it would win

Adding one quarter out of N saves roughly (N - 1)/N of the file reading. FAERS has many years of quarters, so with a long history the saving grows and the fixed cost does not. We expect the trade to turn over somewhere past a handful of quarters ASSUMED; the crossover point is **not measured**.

- Re-measure with 4 or more quarters before switching signals to the incremental outputs (D044).
- Cut the fixed cost (not tried): write the outputs in the same build as the state (one start-up), and keep the state as one table instead of three.

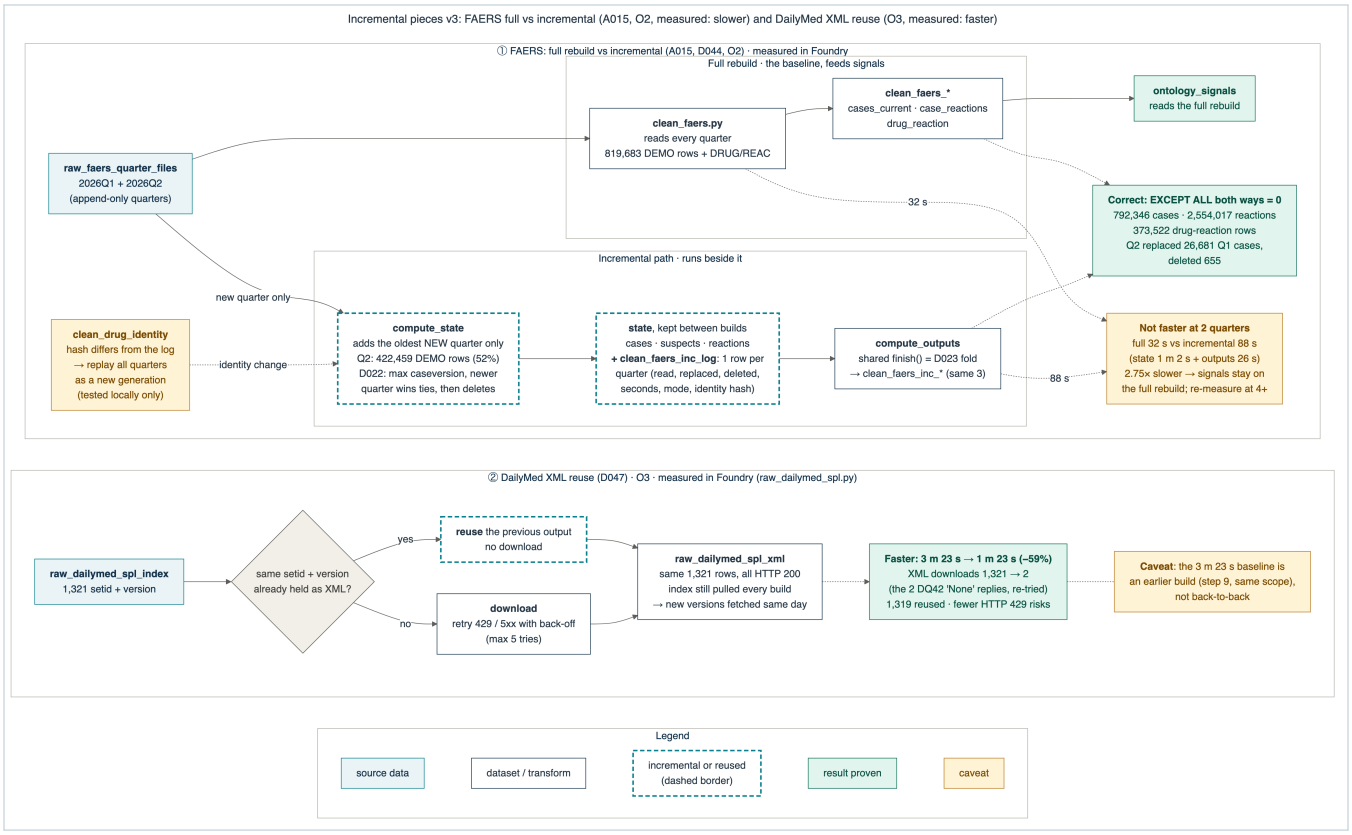


Figure 1. Full vs incremental FAERS (top) and DailyMed XML reuse (bottom), with the measured results.
Source: docs/diagrams/png/33-handin-incremental-faers-v3.png (source: docs/diagrams/src/33-handin-incremental-faers-v3.mmd)

Gate: O2 closed TESTED

Required evidence: Same rows both ways; build time for both paths.

Result: Correct, not faster (2.75 times slower). Signals stay on the full rebuild (D044).

Measure next: 4+ quarters; one-build variant; start-up time on its own.

05

O3 · DailyMed XML reuse

TESTED	Helped: 3 m 23 s to 1 m 23 s (59% less build time), 1,321 to 2 downloads.

The pipeline. `raw_dailymed_spl` first pulls the DailyMed index (which label versions exist for our drugs), then downloads each label's XML. Before this change it downloaded all 1,321 XML files on every build, even though almost none had changed. This was our slowest build. Decision D047 records the change; the owner accepted it on 6 November 2026.

	Value	Evidence
Baseline	Full pull every build: 1,321 XML downloads, **3 m 23 s** (203 s)	OBSERVED
Change	Read the previous output; if the same setid + version is already held as XML, reuse it. Download only new versions and earlier replies that were not XML. The index (about 30 pages, O3) is still pulled every build, and both outputs are still replaced as a full snapshot	DESIGNED
After	2 downloads, 1,319 reused; **1 m 23 s** (83 s); same 1,321 rows	OBSERVED
Reduction	Build time 59% lower; downloads 99.8% lower	DERIVED
Method	Foundry Builds page; reused vs fetched rows in the output	OBSERVED

The 2 remaining downloads are the two labels where DailyMed answers "HTTP 200" with the text "None" instead of XML (data issue DQ42). They are re-tried on purpose, in case DailyMed fixes them.

Side benefits

- **Fewer HTTP 429 replies** ("too many requests"). We saw a burst of them in a local run; fewer calls means less chance of hitting the limit. The pull also retries 429 with back-off.
- **Freshness unchanged.** Because the index is pulled every build, a new label version is still fetched the same day.

Threats to validity

Watch out

The baseline is from an earlier build (the step 9 build, same scope), not run back-to-back with the after build. Platform load on that day may differ.

One run each, wall-clock only. The gap (two minutes) is large enough that we believe the direction, but the exact percentage should be re-measured.

Gate: O3 closed TESTED

Required evidence: Build time and download count before and after, same rows.

Result: Met: 3 m 23 s to 1 m 23 s; 1,319 labels reused.

Measure next: Back-to-back runs (with and without reuse); count of 429 replies per build.

06

O3a · Column pruning

TESTED	No real gain. Reverted.

	Value	Evidence
Baseline	The FAERS clean step parses every column of the case, drug and reaction files, then keeps the few it needs. Whole local run 12.3-13.0 s	OBSERVED
Change	Parse only the needed columns (`read_csv(columns=...)`)	DESIGNED
After	The drug-file parse itself got faster, but the whole local run took 12.2 s; outputs identical (sorted row hashes)	OBSERVED
Method	Local runner timings (not Foundry); the baseline is a range over runs	OBSERVED

Why it did not help: parsing is not the bottleneck. Where the time does go was not profiled. We reverted it so the Foundry code and the local runner stay identical, which matters more for trust than a fraction of a second.

Threat to validity: measured on a laptop, not in Foundry. We did not try it in Foundry because the local gain was already within noise.

Gate: O3a closed TESTED

Required evidence: Before and after timing, same output.

Result: Measured, no gain, reverted. Kept in the log as "what did not help".

Measure next: Nothing; profile where the FAERS build really spends its time first.

07

Deliberate choices

The judges look for deliberate choices on incremental vs full rebuild, partitioning, and not re-running the LLM needlessly. Here is each one, with the reason.

Incremental or full rebuild, per pipeline

Pipeline	Mode	Why
Most raw and clean steps (openFDA recalls, NDC, Drugs@FDA; labels; identity)	Full rebuild	Small (about 50 openFDA requests). Full is simple and always correct.
FAERS clean, used by signals	Full rebuild	Faster at two quarters (32 s vs 88 s, O2). Correct by construction.
FAERS clean, incremental path	Incremental, beside the full one	Proven equal (O2). Switch only when it is measured faster (D044). Replays all quarters if drug identity changes, because old drug matches may then be wrong.
DailyMed raw pull	Full snapshot, reusing held XML	Fast (O3) but keeps the full-snapshot contract: every build lists every label version (D047).
AI label extraction	Incremental (cache)	The expensive step. Never re-call for the same text (O1).
Change feed (`ontology_changes`)	Incremental (keeps last snapshot)	Needs yesterday's state to say "what changed since yesterday".
Monitoring checks	Incremental (appends)	History of checks is the point: a drop is measured against the last build.

Partitioning: we did not partition, on purpose

Partitioning splits a dataset into pieces (for example one per FAERS quarter) so a query or build can skip pieces it does not need. We did not add it DESIGNED:

- **Size.** The largest clean FAERS table has 2,554,017 rows and the full FAERS rebuild takes 32 s. At this size the extra files and tuning cost more than they save ASSUMED.
- **The raw files are already split by quarter.** FAERS lands as one set of files per quarter, and the incremental path reads only the new one. That is the useful kind of split at pilot size.
- **Signals need every case.** The PRR (proportional reporting ratio, the signal statistic) compares our drug with the background of *all* reports, so a split by drug would not let the signal step skip anything.
- **Scoped out on purpose.** The build plan puts partition tuning after the pilot, with its own measurements (project build order v1, section 4).

Decision record

Recorded as decision D048 (accepted 6 Nov 2026): every dataset is written unpartitioned in the pilot. Revisit (partition FAERS by quarter, signals by drug) when FAERS holds 8+ quarters or a single build passes 5 minutes. The same choice is a row in the hand-in log `docs/08-optimization-log/optimization-log.md` (D048). How much of each build is Foundry start-up is not measured separately.

Where the LLM step sits, so it is not re-run

The AI transform sits **after** the labels are cleaned and split into sections, and **before** the signal step. It reads `clean_label_sections` and writes to the curated layer. Its cache key is built only from what changes the answer:

```
cache_key = sha256(setid | version | section_code | sha256(section_text)
                  | parser_version | prompt_version | model_id)
```

- **A label refresh with the same text** gives the same key, so 0 calls (O1).
- **A new label version** or **changed section text** gives a new key, so only that section is read.
- **A new prompt or model** changes the key for every section, so everything is re-read and re-scored against the answer key before it is trusted.
- **Failures** (`parse_error`, `model_error`) are kept visible and retried; they never count as "the label does not mention it".

Gate: Choices are explained TESTED

Required evidence: Each pipeline has a mode and a reason; partitioning and LLM placement explained.

Result: Met (D048, accepted). Partitioning is recorded with its revisit trigger.

If it fails: A reviewer cannot tell a deliberate choice from a default.

08

What we would measure next

Measure	Why	Status
Compute-seconds per build (not just wall-clock)	The brief names compute; wall-clock hides start-up and queueing	Not measured
O2 at 4+ FAERS quarters	Find the point where incremental beats the full rebuild	Not measured
O2 in one build (state + outputs together)	Removes one start-up and one read-back	Not tried
O3 back-to-back, with and without reuse	Remove the earlier-build baseline doubt	Not measured
429 replies per DailyMed build	Show the rate-limit benefit with a number	Not measured
O1 job wall time, like for like	Turn "tokens saved" into "time saved"	Not measured
LLM cost in dollars, per label	Needed before widening the AI past 5 labels to other drugs	Not measured
Foundry start-up time on its own	Explains part of O2's gap	Not measured

Take-away
Two optimizations helped and two did not, and each claim has a before and after number. The biggest lesson: incremental is not automatically faster. It must be measured at the data size you actually have.

Figures quoted from project records (not in numbers.json)

Figure	Source
819,683 DEMO rows; Q2 share 52%; 2.75 times slower; build time 59% lower; downloads 99.8% lower	DERIVED here from two `numbers.json` values each
Prompt p2 used about 55% more output tokens than p1	A011 run 2; DECISIONS learning 2026-11-04
O2 state = three state datasets + one quarter log	`foundry/pipelines/clean/README.md`; O2
Re-measure O2 at 4+ quarters	O2; D044
About 30 DailyMed index pages still pulled every build	O3, optimization log
2 remaining downloads are DailyMed "None" replies	DQ42; O3
O3 baseline from the step 9 build (not back-to-back)	O3, optimization log
Partition revisit at 8+ FAERS quarters or a build over 5 minutes	D048
O1b: 2.3 times the size; Build button skips an unchanged incremental dataset	Optimization log O1b; DECISIONS learning 2026-11-06
Partition tuning after the pilot	project build order v1, section 4

Numbers used in this report

Every number in the table below is looked up from `docs/11-handin/numbers.json` at build time. Source: Foundry SQL (repo SQL panel) after the baseline-gate rebuild of the whole downstream chain: raw_dailymed_spl → clean_labels → clean_label_product_map → ontology_objects → ai_label_extraction → ontology_signals → ontology_changes → monitoring. Build durations from the Foundry Builds page. Query: docs/11-handin/baseline-gate.md. Captured: 2026-11-05T07:50Z. Figures quoted from other project records (decision log, data-issues register, optimization log) name that record where they appear.

Key	Value
ai.d041_v2.cached_sections_in_that_build	9
ai.d041_v2.model_calls	12
ai.d041_v2.new_completion_tokens	26,226
ai.d041_v2.new_prompt_tokens	44,869
ai.d041_v2.semaglutide_labels	3
ai.from_cache_on_rebuild	21
ai.model_calls_on_rebuild	0
ai.pilot_labels	5
ai.run1_tokens_completion	29,072
ai.run1_tokens_prompt	41,704
ai.sections	21
ai.v1_pilot.labels	2
ai.v1_pilot.sections	9
faers.case_reactions	2,554,017
faers.cases_current	792,346
faers.drug_reaction_rows	373,522
faers.q1_cases	397,224
faers.q2_cases_deleted	655
faers.q2_cases_replaced	26,681
faers.q2_demo_rows	422,459
labels.total	1,321
monitoring.faers_age_days	130
optimization.O1.after_calls	0
optimization.O1.after_tokens	0
optimization.O1.before_calls	9
optimization.O1.before_tokens	70,776

Key	Value
optimization.O2.full_rebuild_s	32
optimization.O2.incremental_add_q2_s	88
optimization.O2.outputs_s	26
optimization.O2.state_s	62
optimization.O3.after_downloads	2
optimization.O3.after_s	83
optimization.O3.before_downloads	1,321
optimization.O3.before_s	203
optimization.O3.reused	1,319
optimization.O3a.local_run_after_s	12.2
optimization.O3a.local_run_before_s	12.3-13.0
raw.openfda_requests_approx	50
scope.drugs_formulary	13
scope.glp1	5
scope.opioids	8